CampusFlow Project Guide

Student Activity Management and Reporting System

Project explanation and demonstration manual
Version 1.1.0 | 1 November 2026

CampusFlow centralizes student activities, participation, evidence, approvals, and institutional reporting. This guide explains the implemented application and provides a practical sequence for presenting it to faculty, evaluators, or a project review panel.

The problem we solve

Institutes often maintain event details in separate spreadsheets, photographs in folders, and attendance in individual documents. This creates duplicate records, inconsistent academic classifications, and repeated work whenever a department needs an annual or accreditation report.

CampusFlow records an activity once and reuses its details for student histories, departmental summaries, attendance reports, and academic review documentation. The database and evidence files are stored centrally rather than in the browser.

Project objectives

• Maintain one searchable register across academic years, semesters, departments, classes, and activity categories.

• Connect students, faculty, and external experts to the activities in which they participate.

• Review activities through a controlled workflow and retain the change history.

• Keep reports, photographs, certificates, and attendance sheets with the relevant activity.

• Generate filtered reports and portable backups without recreating the same information.

A short introduction you can speak

“Our project is a centralized Student Activity Management and Reporting System. Faculty record activities and participation, authorized reviewers verify and approve the records, and the institute obtains dashboards and reports from the same data. Students can see their own participation, register for eligible activities, access certificates after completion, and provide feedback.”

Reading route: roles and scope on page 2; architecture on page 3; database on pages 4-5; workflow on page 6; interface on page 7; reports on page 8; security on page 9; setup and testing on page 10; demonstration on page 11; viva questions on page 12.

2 Users and functional scope

The application supports department-level and institute-level use. Account identity and application authorization are separate: sign-in identifies the person, while the stored role and department determine the operations they may perform.

Role	Main responsibilities
Administrator	Manage departments, academic configuration, accounts, all activities, directories, reports, backup, and restoration.
IQAC Coordinator	Review institute-wide activities, verify or approve records, inspect documentation, and generate consolidated reports.
Head of Department	Manage records and students in the assigned department; review departmental submissions.
Faculty Coordinator	Create and maintain their own department activities, add participants, record attendance, upload evidence, and submit records.
Student	View relevant published activities and personal records; register when eligible; access completed participation certificates and feedback.
Management	Read institute-level activities, participation summaries, documentation, and reports without changing records.

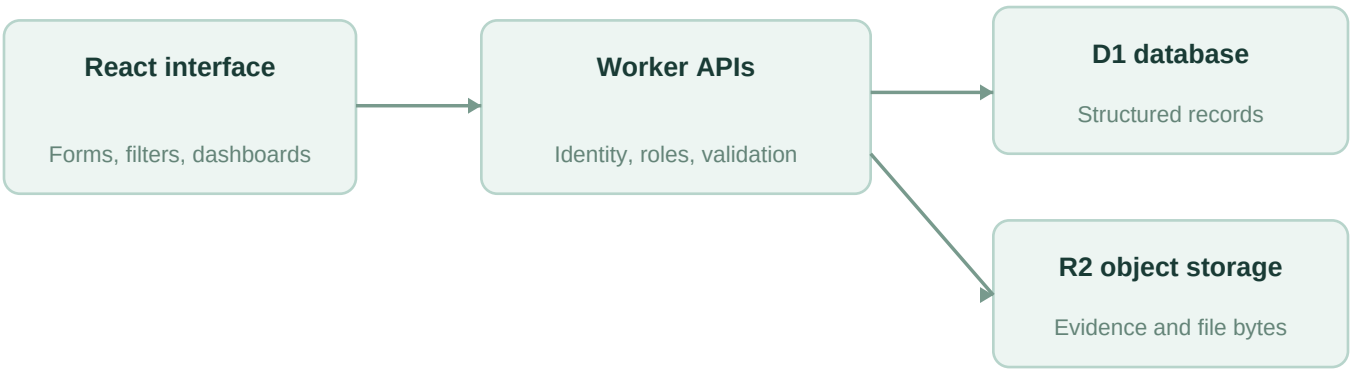
Coverage of the supplied requirements

Requirement group	Implemented behavior
FR 01-02	Authenticated sign-in and server-side role checks. Hosted password/session management is delegated to ChatGPT sign-in.
FR 03-06	Department creation/editing/deactivation; academic years, programs, semesters, classes, divisions, categories, and types; student entry and import.
FR 07-12	Activity entry/editing, participant mapping, attendance, evidence upload, and the approval/correction workflow.
FR 13-16	Search and multi-criteria filters; customized reports; CSV/XLSX exports; browser PDF printing; live charts and statistics.
FR 17-18	Audit and review history plus in-app reminders for pending reviews, returned records, and missing reports. Automatic email is an optional extension.

Present the scope accurately: accreditation selections organize and label institutional reports. They are not pre-certified NBA or NAAC submission forms. Optional QR attendance, email delivery, ERP integrations, scheduled backups, and verified digital signatures are future extensions.

3 Architecture and technology

CampusFlow uses a browser frontend, authenticated application APIs, a relational database, and object storage. The same validation and permission rules apply whether a request comes from a visible button or is submitted directly to an endpoint.



Layer	Technology and purpose
User interface	React 19, TypeScript, custom responsive CSS, and Lucide icons.
Application runtime	Vinext on Vite builds the application for Cloudflare Workers.
Database	Cloudflare D1, using SQLite-compatible SQL and Drizzle ORM/schema migrations.
Evidence storage	Cloudflare R2 stores uploaded files; access is checked by the file API.
Hosted identity	Sites supplies authenticated identity headers and the private Site access gate.
Import and export	Client-side CSV/XLSX utilities; printable HTML for academic summaries and certificates.

Main endpoint responsibilities

Endpoint	Responsibility
/api/manage	Scoped workspace reads, activities, students, attendance, and atomic status transitions.
/api/records	Academic setup, directories, target groups, contributions, account assignments, registration, and feedback.
/api/files	Validate/store evidence and authorize downloads.
/api/backup	Export portable records/files and restore missing records.
/api/certificate	Return a protected, printable participation certificate.

4 Database tables

The implementation has 16 physical tables. Academic lookup entities share **master_data**, while many-to-many relationships use separate junction tables. UUID strings identify individual records; unique business identifiers prevent duplicate PRNs and employee IDs.

Physical table	Important fields and purpose
users	id, email, name, role, department, active. Signed-in application accounts.
account_assignments	email, name, role, department, active. Prepare access rules before first sign-in.
departments	id, code, name, active. Department catalog with a unique code.
master_data	kind, name, parent, department, active. Institute, academic years, programs, semesters, classes, divisions, categories, and types.
students	id, prn, name, email, program, department, class, division, semester, year.
faculty	id, employee_id, name, email, department, designation, active.
resource_people	id, name, designation, organization, email, mobile, address, expertise, active.
activities	id, title, category, type, academic fields, dates, venue, mode, coordinator, objectives, outcomes, budget, sponsor, owner, status.
activity_targets	activity, class, division, program. Additional target groups.
participation	activity, student, attendance, responsibility, achievement, certificate.
activity_faculty	activity, faculty, responsibility. Coordinators, committee members, mentors, judges, and reviewers.
activity_resources	activity, resource, responsibility. Speakers, trainers, judges, and experts.
documents	activity, name, type, MIME, size, storage key, uploaded_by, created.
activity_approvals	activity, reviewer, from_status, to_status, remarks, created. Workflow decisions.
feedback	activity, student, rating, comments, created. One response per activity/student pair.
audit	activity, user, action, remarks, created. Changes, uploads, and workspace operations.

Indexes support department/year and date access. Uniqueness is enforced for department codes, PRNs, employee IDs, participation pairs, feedback pairs, relationship combinations, and new activity duplicate keys. Migrations are appended rather than rewriting already-applied schema history.

5 Relationships and database concepts

Important entity relationships

Relationship	How it is represented
Student many to many Activity	participation links a student to an activity and records attendance, role, achievement, and certificate reference.
Faculty many to many Activity	activity_faculty records multiple organizing roles. A faculty member may contribute to several activities.
Resource person many to many Activity	activity_resources permits several speakers or trainers in one activity and repeated expert contributions.
Activity one to many Evidence	documents maps every uploaded file to its activity; the file bytes are held in R2.
Activity one to many Target groups	activity_targets supports multiple class/division/program combinations.
Activity one to many Reviews	activity_approvals records transitions, reviewer names, and correction remarks.

Why junction tables matter

A student can attend many activities, and an activity can include many students. Storing a comma-separated list of student IDs would make attendance, duplicate detection, and joins difficult. A participation row gives each student/activity pair one well-defined record and lets us attach relationship-specific information.

Foreign keys enforce the central activity/student/faculty/resource relationships. Department codes and academic labels are validated against configuration in the application. The project therefore demonstrates relational modeling and junction tables without claiming that every lookup is a separately normalized SQL table.

Example SQL queries you can explain

```
SELECT department, COUNT(*) AS activity_count
FROM activities
WHERE year = :academic_year
GROUP BY department;

SELECT s.prn, s.name, COUNT(p.id) AS participation_count
FROM students s JOIN participation p ON p.student = s.id
GROUP BY s.id, s.prn, s.name
HAVING COUNT(p.id) > 5;

SELECT a.title FROM activities a
WHERE NOT EXISTS (SELECT 1 FROM documents d
  WHERE d.activity = a.id AND d.type = 'Activity report');
```

Explain the concepts: primary keys identify rows, foreign keys maintain relationships, unique constraints prevent duplicates, joins combine related records, grouping produces summaries, and transactions keep related changes consistent.

6 Complete activity workflow

Stage	User action and resulting behavior
Draft	Faculty creates the activity, records objectives, assigns target groups and organizers, and prepares participation/evidence.
Submitted	The owner submits the record. Core details are locked while the record is reviewed.
Verified	An authorized HOD or institute reviewer verifies the submission. Faculty cannot verify their own record merely by changing the request payload.
Approved	An authorized reviewer approves it. Eligible students may self-register while registration is open.
Rejected for correction	A reviewer supplies a reason. The owner corrects the record and resubmits it; review history is retained.
Completed	The coordinator records outcomes and resolves registered attendance into present/absent before completing the activity.
Archived	A completed activity is retained for reference. Activity and participation records are read-only.

Participation and achievements

The coordinator can select students individually, select a matching class group, import attendance, or mark all present. The same student/activity pair is updated rather than duplicated. Roles include participant, volunteer, winner, and runner-up; achievements and certificate references are stored separately.

Evidence and certificates

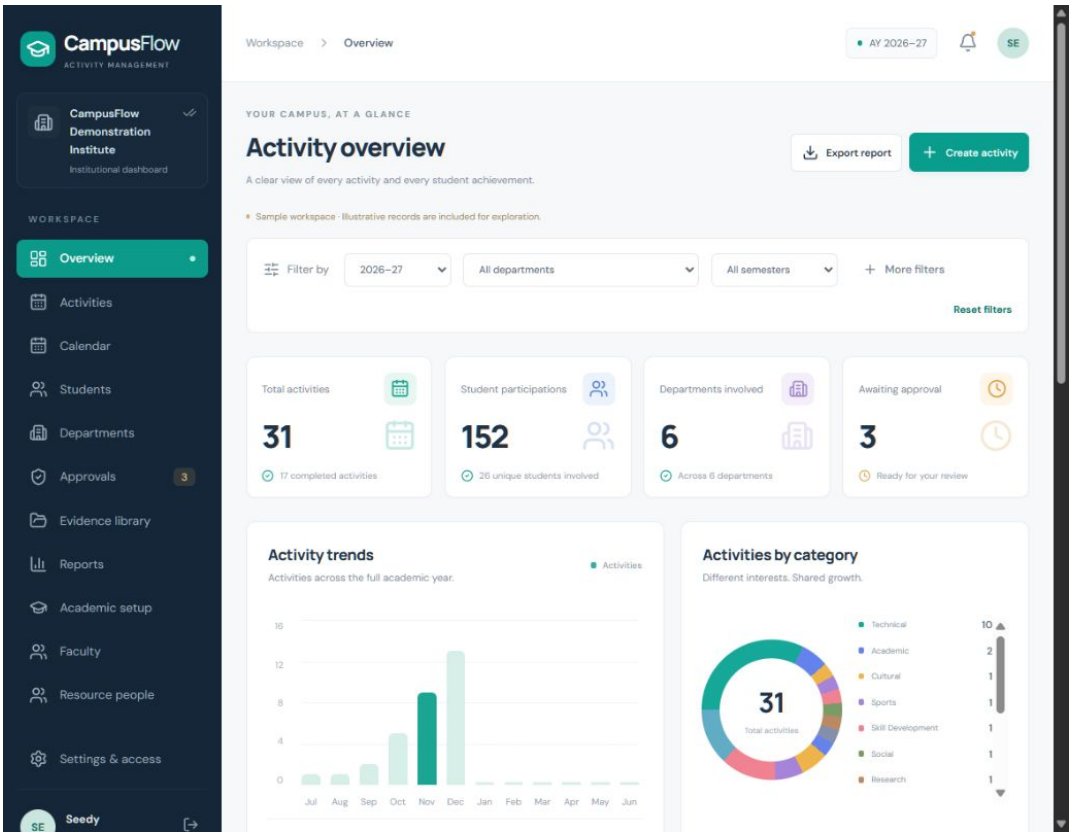
Reports, photographs, attendance sheets, and other evidence are attached to the activity. A present participant in a completed or archived activity can open a generated certificate and use the browser Print command to save it as PDF. The certificate is a participation record; institutional signatures remain the institute’s responsibility.

Feedback and backup

Present participants can submit a rating from 1 to 5 and a comment after completion. Updating feedback changes the existing response. Administrators can export portable backups containing records and evidence bytes. Restoration inserts missing records and files while retaining existing records and account permissions.

Approval updates, review decisions, and their audit entry are committed together. If two reviewers act on the same state concurrently, only one transition succeeds. The second request must refresh.

7 Interface and navigation



Local demonstration workspace after testing. Sample and QA records shown here are illustrative and are separate from the hosted database.

Workspace area	What to demonstrate
Overview and Calendar	Live totals, full academic-year trend chart, category distribution, upcoming activities, and month navigation.
Activities and Approvals	Search/filter records, open the detail tabs, submit a draft, review it, and inspect correction/history entries.
Students and Evidence library	Add/import students, inspect participation histories, record attendance, upload evidence, and download permitted files.
Academic setup and Directories	Maintain institute/academic values, faculty records, experts, and organizing responsibilities.
Reports and Settings	Generate grouped reports, manage accounts, download a backup, restore missing data, and open this guide.

On mobile, the sidebar becomes a menu and summary cards wrap into two columns. Wide data tables scroll inside their panels so the entire page does not overflow horizontally. Dialogs support Escape and keyboard focus trapping.

8 Reports imports and calculations

Report generation

First apply academic year, semester, department, class, division, category, activity type, status, coordinator, student, or date filters. Text search covers activity titles and resource people. Then select the report dimension and export the displayed results.

Report dimension	Purpose
Department / year / semester	Compare activity counts, completed records, participation, attendance, and evidence across academic units or periods.
Class / division / category / type	Review target-group coverage and activity distribution. Additional activity target groups are included.
Student / achievements	Review personal participation, attendance, achievements, certificate references, and activity points.
Coordinator / faculty / expert	Review coordination counts, organizing responsibilities, and external expert contributions.
Missing evidence	Identify activities without an uploaded Activity report for documentation follow-up.

Attendance percentage = present / (present + absent) × 100. Registered rows with unrecorded attendance are excluded from the denominator. Zero recorded attendance produces zero, avoiding division by zero.

Activity points apply only to present participants: participant 2, volunteer 3, winner 10, and runner-up 7. These are demonstration rules and can be changed if an institute has a different policy.

Student and attendance import

Student imports accept CSV or the first worksheet of an XLSX file. Required headers are **name, prn, email, department, class, division, semester, year**; **program** is optional. Attendance imports use **prn, attendance, responsibility, achievement, certificate**. Use the supplied templates and configured department/academic values.

Keep PRNs as text in Excel if they contain leading zeros. Imports accept at most 500 rows and a 10 MB file. Quoted commas are supported in CSV; duplicate headers, malformed rows, and unclosed quotations are rejected.

Export formats

CSV is suitable for data exchange. XLSX exports are actual workbook ZIP/XML files and preserve numeric totals as numeric cells. Academic summaries and certificates use browser Print / Save PDF. IQAC, NBA, NAAC, AAA, and University audit labels help organize review material; institutes must map them to the applicable criterion.

9 Security validation and corrected defects

Permissions are enforced in API handlers, not only through hidden buttons. Student data is matched to the signed-in email. Faculty actions are constrained to owned activities in the assigned department, while HOD actions are scoped to the department.

Area	Validation or correction
Identity and sessions	Hosted identity is provided by Sites. Missing identity is rejected; inactive accounts are denied immediately.
Student privacy	Draft activities, unrelated student records, staff history, and attendance-sheet downloads are excluded from student access.
Academic data	Consecutive academic years, configured classes/divisions/programs, category/type compatibility, and active departments are checked.
Dates and duplicates	Real calendar dates and start/end ordering are validated. New activity duplicate keys prevent concurrent duplicate creation.
Workflow races	Status comparison, approval insertion, and audit recording are atomic. Concurrent reviews cannot create duplicate successful decisions.
Attendance and outcomes	Completion requires outcomes and resolved attendance. Completed/archived participation becomes read-only.
Uploads and downloads	File size/type and PDF/image signatures are checked. Every download checks the current account and activity scope.
Exports and input errors	CSV formula prefixes are escaped, XML controls are removed, malformed JSON is rejected, and request failures are shown to the user.
Startup and refresh	Calendar/year values are date-based. Local startup applies missing schema changes; refresh failures do not claim unsaved changes succeeded.

Controls worth showing during review

Try a reversed date range, duplicate student PRN, duplicate faculty employee ID, spoofed PNG file, and a faculty request to approve an activity. Show that an administrator cannot deactivate or demote their own account. Then show a legitimate activity completing the full path.

The local role-test middleware is explicitly enabled only for development on loopback. It is not a production login feature and is not included as a deployed Worker route. Production remains behind the existing private Site access gate.

10 Setup testing and maintenance

Run locally

Install Node.js 22.13 or newer. Extract the source archive and open a terminal in the project folder. Internet access is needed for the initial dependency installation.

```
npm ci
npm run dev
```

Startup builds the Worker configuration if absent, checks the actual local database schema, applies missing statements, and preserves existing records. Open the Local URL printed by the server, normally **http://127.0.0.1:5173**. The loopback sign-in uses a development account. Hosted sign-in uses ChatGPT.

Verification commands

```
npx tsc --noEmit
npm run build
node scripts/test-exports.mjs
# With the ordinary development server running
npm run test:integration
```

For the six-role end-to-end suite, stop the ordinary server and start explicit role-test mode in one terminal. Run the test in a second terminal.

```
npm run dev:test
# In a separate terminal
npm run test:requirements
```

Evidence	What was verified
Baseline API suite	Nine grouped checks cover sign-in protection, validation, duplicates, students, attendance, files, review flow, and persistence.
Extended requirement suite	Fifteen grouped checks cover six roles, configuration, scopes/privacy, relationships, concurrent requests, registration, certificates, feedback, and real restoration.
Export and browser checks	CSV edge cases, XLSX ZIP/XML integrity and numeric cells; browser form/configuration checks, compressed Excel import, reports, and responsive navigation.
Build checks	TypeScript type checking and the production build. Re-run after changes to application code or schema.

Local records and R2 bytes are under ignored .wrangler/state/v3. Hosted D1/R2 are separate. Keep backups securely; do not commit credentials or test database files. Portable backups support 5000 records and 20 MB of evidence, and restore only missing records.

11 Demonstration and presentation script

Prepare departments and academic values before presenting. Use a small, clearly labeled demonstration dataset, two student records, one faculty member, one expert, and a short PDF/CSV evidence file. Keep the activity date open for registration and use a unique title.

Time	Demonstration step
0:00-0:45	Explain the manual spreadsheet/folder problem and show the overview dashboard.
0:45-1:30	Open Academic setup. Explain department codes, academic years, categories, and activity types.
1:30-2:30	Create a faculty-owned workshop with objectives, dates, venue, coordinator, budget, and sponsor.
2:30-3:15	Use Organization to add a target group, faculty responsibility, and resource person.
3:15-4:15	Submit as faculty, verify as HOD, and approve as IQAC. Show the review history.
4:15-5:15	Register an eligible student. Add/record attendance and upload supporting evidence.
5:15-6:00	Record learning outcomes, complete the activity, and open the student certificate and feedback form.
6:00-7:00	Filter reports by department/year. Show student or faculty grouping and export Excel or PDF.
7:00-8:00	Show backup download, explain non-overwriting restoration, and finish with one validation example.

Role testing for a local demonstration

After starting **npm run dev:test** and running the extended suite, local test profiles can be selected through **/__test__/signin?profile=** followed by **admin**, **faculty**, **hod**, **iqac**, **student**, or **management**. These are development profiles only; the hosted Site uses real permitted accounts.

A closing explanation you can speak

“The key contribution is that activity details, student participation, and evidence are connected. One activity record drives several reports, while role checks and review history make changes accountable. The project demonstrates database relationships, validation, file management, transactions, and a complete web application workflow.”

Use the database page when asked about keys or relationships, the security page when asked about access control, and the testing page when asked how the implementation was verified.

12 Viva questions and project boundaries

Why use a centralized system?
It reduces repeated entry and inconsistent folders, and lets departments generate different reports from the same activity records.

What is the main many-to-many relationship?
Students and activities are linked through participation. That row also stores attendance, participation role, achievement, and certificate reference.

How are duplicate activities prevented?
The server compares normalized title, department, and start date; a unique duplicate key also protects concurrent creation requests.

How is authentication different from authorization?
Authentication establishes identity. Authorization checks the stored role, active status, department, ownership, and operation being requested.

Why separate database and file storage?
D1 holds searchable structured records. R2 holds file bytes. A document row connects its file to the activity.

What prevents two reviewers approving the same state?
The transition compares the expected current state and commits the update, decision record, and audit entry together.

Why choose SQLite-compatible D1 rather than MySQL?
This implementation targets the Sites Worker runtime and its managed D1 binding. The relational model can be adapted to another database.

Can a student access everyone’s attendance?
No. Student workspace reads are scoped to personal records, and attendance-sheet downloads require staff access.

Does backup restoration overwrite existing data?
No. It restores missing records and evidence bytes, preserves existing records, and does not restore account permissions.

What is the difference between approval and completion?
Approval authorizes the activity. Completion records the delivered outcomes and final attendance after the activity has taken place.

Boundaries and future improvements

The release includes student registration, feedback, activity points, and printable certificates. QR-based attendance, automatic email notifications, scheduled backup jobs, ERP integration, verified digital signatures, and institution-specific accreditation forms remain extensions. Generated certificates are participation records and require the institute’s normal approval/signature process where applicable.

The source archive contains the application, schema migrations, setup scripts, verification scripts, and README. This guide is also available from Settings in the application. Hosted data is not bundled with the source archive.